

MAS Platform Guide

Human-oriented companion to the authoritative platform specification

MARCH 26, 2026

Contents

MAS Platform Guide

How To Read This Guide

Five Nouns To Keep Separate

1. Platform Identity And Vocabulary

Core Vocabulary

Three Execution Types

Flow And State Basics

2. The Core Mental Model

The Runtime In Six Sentences

3. Flow Packages And Contract Files

Canonical Directory Layout

Minimum Package Rules

Loader Defaults

Entity Schema And Event Schema

Persistence Tools Derived From Entity Schema

Required Agents

Underscore-Prefixed Contract Fields

4. The Flow Model

What A Flow Declares

Nesting

Pins: The Public Interface Of A Flow

Cross-Flow Event Ownership

Stateless Flows

State Composition Across Flows

5. Hello World Example

6. Handler Execution

The Dependency Graph

Atomicity

Short-Circuits

`on_complete` And `rules` : Two Ways To Branch

Core Fields vs. Extension Fields

Handler Fields At A Glance

The Only Valid Actions

Canonical `data_accumulation`

Rules And Handler-Level Defaults

7. The Engine Runtime

Flow Tree Walker

URI Addressing

Contract Merger

Static Flows, Template Flows, And Dynamic Instances

Boot Sequence

Event Loop

State Management

Accumulation Engine

Agent Session Management

Timer Model

Expression Language

Error Model

Boot Verification

8. Compliance, Permissions, Tools, And Prompts

Compliance

Permissions Model

Tool Model

Prompt Templating

9. Versioning

10. Platform Tables And Runtime Data

The Table Set

What Each Table Is For

Where To Start

11. Observation Model

Emitted Events

Payload Construction

Entity State Observation

Handler Outcome

12. Workspace Model

Standard Mounts

Mount Guarantees

Deployment Mapping

13. Crosswalk To The Reference Spec

14. Recommended Documentation Split

15. Appendix A: Runtime Story

What This Runtime Story Is Saying

The Minimal Mental Shortcut

16. Appendix B: Contract Author Checklist

Package-Level

Schema-Level

Events

Nodes

Handlers

Agents

Prompts And Policy

Runtime Behavior

17. Appendix C: What Changes Where?

Practical Authoring Order

The Common Misplacements

18. Appendix D: State And Storage Cheat Sheet

“What state does an entity have?”

“Where do I look for event history?”

“Where do I look for flow routing?”

“Where do I look for sessions and agents?”

“Where do I look for human tasks?”

“Where do I look for spend?”

“Where do I look for dynamic flow instances?”

“Where do I look for timers?”

MAS Platform Guide

This guide is the human-oriented companion to [platform-spec.md](#). It covers the same platform surface but organizes the material around how a reader usually learns and uses the platform, rather than around the raw contract layout.

Authority boundary. The authoritative source of truth is always [platform-spec.yaml](#). The prose spec in [platform-spec.md](#) is the exact rendering of that YAML. This guide is explanatory only. If this guide and the spec ever disagree, the YAML and spec win.

How To Read This Guide

The guide is written in a learning order:

1. Start with the platform mental model — what the platform *is* and what the key nouns mean.
2. Read the contract package model and the flow model together — these define how you author things.
3. Read handler execution and engine execution as one runtime story — this is how authored contracts come to life.
4. Read permissions, tools, prompts, compliance, and workspaces as the control surface around that runtime.
5. Use the platform tables and observation model as the “what can I inspect?” layer.

The guide keeps exact identifiers, enum values, and technical terms from the spec. Where the spec is dense, the guide explains how adjacent sections fit together.

FIVE NOUNS TO KEEP SEPARATE

Most onboarding confusion comes from mixing these five concepts together. Before you go further, make sure they are distinct in your head:

A **flow** is a reusable package definition — a directory on disk containing contract files that describe a piece of work. Think of it as a blueprint. It defines states, events, agents, and how they connect.

A **flow instance** is one running copy of that flow. A flow called `scoring` might produce many scoring instances at runtime, each tracking a different piece of work independently. The flow is the class; the instance is the object.

An **entity** is the thing moving through a flow instance’s state machine. It has a current state, gate flags, and accumulated data fields. Each entity belongs to exactly one flow instance and progresses through that instance’s states.

An **agent** is a runtime role with subscriptions, tools, and permissions. It is defined in `agents.yaml` — its identity, what events it listens to, what tools it can use, and what model tier it runs on.

An **agent session** is one execution context for an agent, scoped by `conversation_mode`. In `task` mode, every event creates a fresh session. In `session_per_entity` mode, the agent keeps continuity across events for the same entity. The agent is the role definition; the session is one active invocation of that role.

1. Platform Identity And Vocabulary

The platform is `mas-orchestrator` at version `1.3.0`. Everything in this guide describes behavior at that version.

The platform vocabulary defines the basic terms used throughout all contracts and the runtime. The most important ones are covered below; the full list is in the spec.

CORE VOCABULARY

A **state_change** is a transition in the entity state machine, triggered by the `advances_to` field in a handler.

A **guard** is a boolean check evaluated before handler execution. Guards use CEL expressions evaluated against `entity`, `payload`, and `policy` context. If a guard fails, the handler is blocked and the configured `on_fail` action runs. The possible `on_fail_actions` are:

- `reject` (the default) — refuse the event
- `discard` — silently drop it
- `kill` — move the entity to a terminal state
- `escalate:{event}` — emit a named escalation event

An **action** is a platform-provided operation that runs as the final step of a handler. Only two valid actions exist: `create_flow_instance` and `record_evidence`. There are no product-specific actions.

A **timer** is a time-based trigger attached to a stage. When its delay elapses, the platform emits the timer's configured event, which enters the normal event loop like any other event. Timers are declared with these required fields: `id`, `state`, `event`, `owner`. Optional fields include `delay_seconds`, `delay_minutes`, `delay_hours`, `delay_days`, `cancellation`, and `recurring`.

THREE EXECUTION TYPES

The platform distinguishes three kinds of runtime actors. Understanding this distinction is important because it determines what an actor can do and how it is declared.

A **system_node** is deterministic code. No LLM. It subscribes to events, executes fixed logic, emits events, and owns workflow transitions. System nodes are the backbone of orchestration.

An **agent** is LLM-powered. It has a system prompt, a model tier, and a conversation mode. It subscribes to events, reasons about them, and calls tools. Agents can own transitions where the decision requires judgment.

The **runtime** is the platform itself. It handles transitions that are implicit — human decisions, system lifecycle behavior — without needing to be declared in any contract.

FLOW AND STATE BASICS

A **flow** is a self-contained unit with input and output pins, system nodes that orchestrate transitions, and required agent roles. The required contract files named in the vocabulary are `nodes.yaml`, `events.yaml`, and `schema.yaml`.

A **state** always belongs to a workflow. An entity is always in exactly one state. States belong to phases for grouping, and some states are terminal (no outgoing transitions). The required state fields are `id` and `phase`. An optional `description` field is also available.

2. The Core Mental Model

The easiest way to understand the MAS platform is to think in layers. At the bottom are contract files that define what a flow does. On top of those, the engine loads, validates, and runs everything.

The contract layer has these files:

- `schema.yaml` defines the flow's public interface — its states, pins, and required roles.
- `nodes.yaml` defines deterministic orchestration — system nodes and their handlers.
- `agents.yaml` defines LLM workers and coordinators.
- `events.yaml` defines event payload shapes.
- `tools.yaml`, `policy.yaml`, and `prompts/` define the operating environment for agents.

The engine layer loads all flows, resolves their paths, validates them against every contract rule, starts subscribers, and runs the event loop.

THE RUNTIME IN SIX SENTENCES

Here is the thread connecting the rest of the spec:

1. A flow declares states, pins, and required roles.
2. System nodes subscribe to events and own deterministic transitions.

3. Agents subscribe to events and handle reasoning work.
4. Events move work through the system — they are the only communication mechanism.
5. Each handler execution commits atomically.
6. The platform persists state, events, deliveries, receipts, sessions, timers, and other runtime infrastructure in platform-owned tables.

3. Flow Packages And Contract Files

Every flow uses the same contract layout. `package.yaml` is the manifest. `schema.yaml` is always required. The remaining files are present when the flow needs them.

CANONICAL DIRECTORY LAYOUT

```
{root_flow}/
  package.yaml      # Root flow (entry point)
  schema.yaml       # Manifest: name, version, flows, handoffs, entity_schema
  nodes.yaml        # Root flow pins
  events.yaml       # Root-level system nodes
  agents.yaml       # Root-level events
  tools.yaml        # Root-level agents
  policy.yaml       # Shared tools
  prompts/         # Shared policy
  runtime/         # Root-level agent prompts
  nodes.yaml        # Merged flat files for flat file loader
  events.yaml
  agents.yaml
  tools.yaml
  policy.yaml
flows/
  {child_flow}/     # Child flow (same structure, recursive)
    package.yaml
    schema.yaml
    nodes.yaml
    events.yaml
    agents.yaml
    tools.yaml      # Optional
    policy.yaml     # Optional
    prompts/
    flows/         # Grandchild flows (if any)
      {grandchild}/
        package.yaml
        ...
```

```
platform/                                # Platform contracts (separate from any flow)
  package.yaml
  platform-spec.yaml
```

MINIMUM PACKAGE RULES

The minimum flow package requires only `package.yaml` and `schema.yaml`. Everything else is optional and follows a simple logic:

- No agents means no `prompts/`.
- No handlers means no `nodes.yaml`.
- No events means no `events.yaml`.

The optional files are `nodes.yaml`, `events.yaml`, `agents.yaml`, `tools.yaml`, and `policy.yaml`. The optional directories are `prompts/` and `flows/`.

LOADER DEFAULTS

Several fields are optional in contract YAML because the loader derives them from context:

- `schema_name` — derived from the directory name if omitted. A flow at `flows/scoring/` gets name `"scoring"`.
- `schema_namespace` — derived from the flow path relative to root.
- `agent_id` — derived from the YAML map key. In `agents.yaml`, `{"opco-ceo": {model_tier: sonnet, ...}}` means `agent_id = "opco-ceo"`. An explicit `id` field overrides the map key if present, but this is discouraged.
- `agent_emit_events` — defaults to an empty list. An agent that only observes may omit `emit_events` entirely.

ENTITY SCHEMA AND EVENT SCHEMA

These two schemas serve very different purposes and are easy to confuse.

Entity schema (`entity_schema` in `package.yaml`) declares the persistent fields for entities managed by this flow. The platform derives database DDL from it at boot. The schema is organized into named groups, and the supported types are: `text`, `integer`, `numeric(precision,scale)`, `boolean`, `jsonb`, `timestamp`, `uuid`.

Event schema (`events.yaml`) defines the payload shape for each event. It tells the platform what fields an event carries so it can validate emit calls at runtime.

Here is the critical distinction: `events.yaml` defines payload *shapes*. It does **not** define routing. Routing is derived from:

- `agents.yaml` — subscriptions and `emit_events`
- `nodes.yaml` — `subscribes_to` and `event_handlers`

The routing consequences are:

- If a system node subscribes to an event, that node owns it.
- If both a node and agents subscribe, both receive it (dual delivery).
- If only a node subscribes, it is node-only delivery.
- If no node subscribes, it is pure agent delivery (passthrough).

This separation means you can change who receives an event by editing subscriptions, without touching the event’s payload schema.

PERSISTENCE TOOLS DERIVED FROM ENTITY SCHEMA

Agents do not get raw database access. Instead, the platform auto-generates typed persistence tools from `entity_schema`:

- `get_entity` — read an entity by ID
- `save_entity_field` — write a specific field on an entity
- `search_entities` — query entities by stage, field values, or metadata
- `query_metrics` — read aggregated metrics across entities

REQUIRED AGENTS

`required_agents` in `schema.yaml` names the roles a flow requires. The fulfillment rule is strict: the role name in `required_agents` must exactly match the top-level agent key in `agents.yaml`. There is no separate `fulfills:` field.

The spec also allows “empty” required agent declarations. A role may exist with empty subscriptions and emits if it coordinates through universal tools like `agent_message` and `mailbox_send`. This is common for managerial roles that coordinate via messages rather than event subscriptions.

UNDERSCORE-PREFIXED CONTRACT FIELDS

The contracts use two categories of `_`-prefixed fields:

Semantic fields that the platform and verifier understand: `_source`, `_consumer`, `_status`, `_producer`. These suppress verifier warnings (like NO-PRODUCER or NO-CONSUMER) and carry meaning.

Documentation-only fields that the platform ignores: anything else starting with `_` (like `_note`, `_routing_note`, `_internal_events_note`). These are human comments. The verifier skips them. They are not part of the contract.

4. The Flow Model

The spec says a `Flow` is the universal building block. There is no separate “project” concept. Every level of the system is a flow, from the root flow down to the smallest sub-flow.

WHAT A FLOW DECLARES

A flow package is defined by its directory of contract files. `package.yaml` and `schema.yaml` are always required. Beyond those, you add files based on what the flow does: `agents.yaml` when the flow has agents, `events.yaml` when it declares event schemas, `nodes.yaml` when it has system nodes, `tools.yaml` and `policy.yaml` when it needs flow-specific configuration, `prompts/` when it has agent prompts, and `flows/` when it has child flows.

NESTING

Flows nest recursively. A flow’s `package.yaml` names child flows in its `flows:` list. Each child lives under `flows/` and has the same structure. The root flow is not structurally special — it is the entry point, but it is still just a flow with nodes, agents, events, policy, and child flows.

The practical way to think about nesting:

- The root flow is the top of the address tree.
- Child flows are named subdomains under that root.
- Each nested flow keeps its own local contract files.
- The engine walks the tree and turns that nested structure into one runtime registry.

So the filesystem is nested for authors, but the runtime turns it into one connected event system.

PINS: THE PUBLIC INTERFACE OF A FLOW

Pins are what the outside world is allowed to assume about a flow. They are its public API.

- **Input event pins** are the events the flow subscribes to from the outside.
- **Output event pins** are the events the flow emits to the outside.
- **Input data pins** are the entity fields the flow reads.
- **Output data pins** are the entity fields the flow writes.

The spec enforces these constraints at boot:

- Required input event pins must be wired (some other flow must emit them).
- Required input data pins must be available from earlier outputs or initial data.
- No two flows may write the same output data pin.

An important subtlety: `events.yaml` can contain more events than the pin list shows. That is because a flow may have **internal events** — events used inside the flow's own orchestration that are not part of its public interface. The pin list is the public contract; `events.yaml` is the full implementation inventory.

CROSS-FLOW EVENT OWNERSHIP

Event schemas are defined once, in the flow that emits them. Consuming flows reference them by absolute path. They do not redefine them. If the same event name appears in multiple flow event files, the emitter flow's definition is authoritative and boot logs a warning.

STATELESS FLOWS

The spec explicitly allows stateless flows. In a stateless flow, `initial_state` is `null`, `states` is empty, no `entity_state` rows are created, `advances_to` is invalid, guards may reference `payload` and `policy` but not `entity`, and accumulators are flow-scoped rather than entity-scoped.

STATE COMPOSITION ACROSS FLOWS

Each flow has its own state machine. There is no single global state enum for an entity across the entire platform. When an entity moves from one flow to another, one flow emits a cross-flow event and the next flow handles that event into its own state space.

5. Hello World Example

The spec's hello world example is the smallest useful illustration of the contract model. It shows one flow with one event, one handler, and one state transition.

package.yaml:

```
name: hello
version: 1.0.0
platform_version: ">=1.1.0"
flows: []
```

schema.yaml:

```

initial_state: waiting
terminal_states: [done]
states: [waiting, done]
pins:
  inputs:
    events: [hello.requested]
  outputs:
    events: [hello.completed]

```

nodes.yaml:

```

hello-node:
  id: hello-node
  execution_type: system_node
  subscribes_to: [hello.requested]
  produces: [hello.completed]
  event_handlers:
    hello.requested:
      advances_to: done
      emits: hello.completed

```

events.yaml:

```

hello.requested:
  payload:
    entity_id: string
    message: string
hello.completed:
  payload:
    entity_id: string

```

This example shows the core platform loop in one handler: receive an event, advance state, emit a follow-up event. Every handler in the platform, no matter how complex, is a variation on this pattern.

6. Handler Execution

The handler specification is one of the most important parts of the platform. If you understand handlers, you understand how all work actually happens.

A common misconception is that the spec defines a 10-step sequential recipe. It does not. It defines a **dependency graph**. The graph says which steps must finish before other steps can begin. Steps at the same level are independent — the engine may execute them in any order.

THE DEPENDENCY GRAPH

This graph is the central execution model:

```

query (optional – pre-fetch cross-entity data into handler context)
  ↳ clear_gates (optional – reset gates before guard)
    ↳ guard
      ↳ accumulate
        ↳ filter (optional – prune accumulated items)
          ↳ reduce (optional – aggregate filtered items to single value)
            ↳ count (optional – count items)
              ↳ compute
                ↳ on_complete / rules (mutually exclusive)
                  ↳ advances_to ⊥
                    sets_gate ———| (independent of each other)
                    data_accumulation ⊥
                      ↳ payload_transform
                        ↳ emits
                          ↳ action
                            ↳ clear (optional – reset
accumulator/state buckets)

```

Arrows mean “must complete before.” That is why the spec says YAML field order within a handler is cosmetic — the engine executes fields per this graph, not per source order. Authors *should* order fields to match the graph for readability, but the engine does not require it.

ATOMICITY

Every handler execution is atomic. State changes, gate updates, data writes, and event persistence all commit in a single database transaction. No external observer ever sees intermediate state.

An important consequence: guards evaluate against entity state *before* any handler writes. The current handler’s `data_accumulation` does not affect the current handler’s guard. It affects the *next* handler that runs after this transaction commits.

SHORT-CIRCUITS

Handlers can stop early in two important ways:

- **Guard failure:** The handler stops and runs the configured `on_fail` behavior (`reject`, `discard`, `kill`, or `escalate:{event}`). No further steps execute.

- **Accumulate incomplete:** The handler records the event arrival and stops. No further steps execute until the accumulation completion condition is met.

ON_COMPLETE AND RULES: TWO WAYS TO BRANCH

These are mutually exclusive — a handler uses one or the other, never both.

on_complete is an ordered list of `{condition, advances_to, emits}` objects. Conditions are evaluated top to bottom; the first match wins. This is used after accumulation or computation when you need conditional branching based on computed results.

rules is a map of named rules used for payload-based routing. Each rule has a condition evaluated against the payload, plus optional `emits`, `advances_to`, and `data_accumulation`. This is used for type-dispatching — routing different kinds of incoming events to different outcomes.

CORE FIELDS VS. EXTENSION FIELDS

Most handlers use a small core subset. The spec draws this distinction explicitly.

Core fields (used by 90%+ of handlers — learn these first): `guard`, `advances_to`, `sets_gate`, `data_accumulation`, `emits`, `rules`, `on_complete`

Extension fields (used by specialized handlers — learn when needed): `accumulate`, `compute`, `fan_out`, `filter`, `reduce`, `count`, `query`, `clear`, `payload_transform`, `clear_gates`, `action`

HANDLER FIELDS AT A GLANCE

Field	Purpose
<code>description</code>	Documentation only. Not executed.
<code>guard</code>	Gatekeeper — blocks execution if conditions fail.
<code>accumulate</code>	Wait for multiple event arrivals before continuing.
<code>compute</code>	Run a platform computation primitive (<code>weighted_average</code> , <code>sum</code> , etc.).
<code>on_complete</code>	Post-accumulation/computation conditional branching.
<code>advances_to</code>	Set <code>entity.state</code> to a target state.
<code>sets_gate</code>	Set <code>entity.gates.{name} = true</code> .
<code>data_accumulation</code>	Write payload-derived values into entity state.
<code>emits</code>	Publish follow-up event(s).
<code>rules</code>	Payload-based routing with per-rule side effects.
<code>fan_out</code>	Emit one event per item in a list.
<code>query</code>	Pre-fetch cross-entity data into handler context.

Field	Purpose
reduce	Aggregate accumulated items into a single value.
filter	Keep only items matching a condition.
count	Count items matching a condition.
clear	Reset accumulator or state buckets after everything else.
action	Invoke a platform action (<code>create_flow_instance</code> or <code>record_evidence</code>).
payload_transform	Construct emitted payloads explicitly from multiple sources.
clear_gates	Reset all entity gates to false.
evidence_target	Required when action is <code>record_evidence</code> .

THE ONLY VALID ACTIONS

The platform has exactly two valid handler actions:

`create_flow_instance` creates a new dynamic flow instance from a template. It requires three sibling fields: `template` (the template flow ID), `instance_id_from` (the payload field containing the unique instance ID), and `config_from` (maps instance variable names to payload field paths).

`record_evidence` appends event payload data to the target accumulator. It requires `evidence_target`.

CANONICAL DATA_ACCUMULATION

The spec defines one canonical shape for `data_accumulation`:

```
data_accumulation:
  writes:
    - field_name           # direct: payload.field_name → entity.field_name
    - source_field: payload_key   # mapped: payload.payload_key → entity.target_key
      target_field: entity_key
    source_event: event_name     # optional: defaults to the trigger event
```

Computed writes are also supported:

```
data_accumulation:
  writes:
    - target_field: entity_key
      expression: "entity.counter + 1"  # CEL expression → entity.target_key
```

RULES AND HANDLER-LEVEL DEFAULTS

When a handler has both `rules` and handler-level fields, the interaction is:

1. The matching rule fires first with its condition-specific side effects.
 2. Handler-level `data_accumulation` executes after rules — it *supplements* rule-level writes, it does not replace them.
 3. Handler-level `emits` fires after rules — both rule-level emits and handler-level emits fire (handler-level is unconditional).
 4. For fields like `advances_to` and `sets_gate`, the rule value *overrides* the handler-level value if present. Fields not specified in the rule fall through to handler-level.
-

7. The Engine Runtime

The engine defines how flows are discovered, loaded, validated, started, and executed. This section tells the runtime story from boot to steady-state event processing.

FLOW TREE WALKER

The engine starts by discovering the flow structure. The flow tree walker begins at the root directory, reads `package.yaml`, registers the flow, descends through `flows:`, and produces a flat list of flow entries.

Important constraints:

- `package.yaml` and `schema.yaml` are always required.
- A flow must have either `agents.yaml` or at least one child flow.
- Maximum nesting depth is `99`.
- Duplicate flow IDs anywhere in the tree are a boot error.

The important onboarding point is that the engine first discovers structure and only then validates meaning. It does not try to partially execute contracts while still discovering them.

URI ADDRESSING

The engine gives every addressable runtime entity — every node, agent, and event — a path. Two forms matter day to day:

- **Local:** `{name}` — means “the thing called this in the current flow.”
- **Absolute:** `{flow_id}/{flow_id}/.../{name}` — means “the thing at this specific location from the root.”

The rule is simple: no slash means local, slash means absolute. So `portfolio-node` is “the node in my flow,” while `validation/research/research.completed` is “the `research.completed` event inside the `research` flow inside the `validation` flow.”

The path is not decoration. It is how the platform avoids global name collisions. Two flows can each have a node called `orchestrator` and there is no conflict — they live at different paths.

What * And ** Mean In Subscriptions

This is one of the places where the spec is compact and first-time readers benefit from a more explicit explanation.

***** **matches one path segment** at that position. Think of it as “one hop.” So `*/entity.shortlisted` means “match `entity.shortlisted` from any direct child flow under the current level.”

****** **matches across any depth** below that point. Think of it as “any number of hops.” So `**/entity.completed` means “match `entity.completed` anywhere below this point, even through multiple levels of nested flows.”

These wildcard patterns are used in subscription matching across flow instances. They are not a separate routing system — they are a way to write subscriptions that match events from flows that may not exist yet (like dynamic instances).

Local Versus Absolute References

When authoring contracts, prefer local names when the target is inside the same flow and absolute paths when the target is in another flow. This keeps contracts readable and lets the platform resolve everything into full internal paths at boot.

CONTRACT MERGER

After flow discovery, the engine builds runtime registries. Every node, agent, and event receives a full URI. Tools are flat and shared (root-level tools are available to all children; a child tool with the same ID overrides the root tool within that child). Policy is hierarchical, with child values overriding parent values for the same key.

The useful consequence is that authors can reuse common local names like `orchestrator`, `ready`, or `completed` in different flows without creating runtime ambiguity. The full path makes each one unique.

STATIC FLOWS, TEMPLATE FLOWS, AND DYNAMIC INSTANCES

This is another area that benefits from a clear explanation of three distinct concepts:

A **static flow** is declared in `package.yaml` with `mode: static` (the default). The platform creates one running instance at boot. It has a fixed path like `{flow_id}/{local_name}`.

A **template flow** is declared with `mode: template`. The platform loads its definition at boot but does *not* start a running copy. It is a blueprint waiting to be instantiated.

A **dynamic instance** is a running copy created from a template at runtime. A handler calls `create_flow_instance`, the platform validates the template and `instance_id`, loads the template contracts, registers nodes, agents, and events, creates the entity at the flow's `initial_state`, expands wildcard subscriptions, and starts the new instance. Its path includes the instance ID: `{template_id}/{instance_id}/{local_name}`.

That means a template can produce many sibling instances, each with its own path, entity state, subscriptions, and runtime lifecycle.

Why Wildcard Expansion Matters For Dynamic Instances

Suppose an agent wants to observe events from every instance under a template. The agent cannot hardcode future instance IDs, because those instances do not exist yet. A wildcard subscription like `{template_id}/*/entity.completed` solves that. When a new instance is created, the platform expands existing wildcard subscriptions so the new instance becomes visible automatically.

That is why the spec talks about wildcard subscriptions being updated on dynamic instance creation — it is the mechanism that makes dynamic flows observable.

BOOT SEQUENCE

The boot sequence moves from `load_platform_spec` to `ready` through 15 ordered steps. The most important checkpoints are:

1. Walk the flow tree.
2. Construct paths.
3. Register templates (but do not start them).
4. Build registries (nodes, agents, events, tools, policy).
5. Resolve subscriptions (local, absolute, and wildcard).
6. Validate: pins, required agents, tools, permissions, platform version.
7. Initialize state stores.
8. Start system nodes, then start agents.

If any boot step fails, startup aborts completely. There is no partially-valid runtime where some flows started and others did not. That “all or nothing” behavior is one of the platform’s strongest operational guarantees.

EVENT LOOP

Once boot completes, the platform enters the event loop. This is the core runtime cycle:

1. **Event arrives** — from an agent emit, system node emit, timer fire, human input, handoff, or external API.
2. **Payload validation** — checked against `events.yaml` schema. Invalid events are rejected.
3. **Event persisted** — written to the event store before any processing begins. If the platform crashes after this point, the event replays on recovery.
4. **Subscribers resolved** — system nodes (from `subscribes_to`) and agents (from subscriptions) are identified.
5. **Node handling** — for each subscribing system node: acquire entity lock, execute the dependency-graph handler atomically, release lock. Emitted events are persisted inside the atomic boundary but delivered *after* commit.
6. **Agent delivery** — for each subscribing agent: the event is placed in the agent’s inbox. Processing is asynchronous.
7. **Emitted events delivered** — events emitted by node handlers in step 5 now enter the loop at step 1 in their own cycle.
8. **Original event marked delivered.**

Two implications are especially important:

Per-entity serialization. Events for the same entity are processed one at a time. This means the platform protects entity state from races without requiring flow authors to reason about low-level locking. Events for *different* entities are fully concurrent.

Post-commit delivery. A node handler never recursively “dives into” the next emitted handler before its own transaction is complete. This keeps handler chains understandable and prevents deadlock-style behavior.

STATE MANAGEMENT

The engine tracks three kinds of state per entity instance. Understanding the distinction helps you reason about contracts:

entity_state answers “where is this entity in the flow?” It is a single string value set by `advances_to`. Initial value comes from `schema.yaml`, terminal values from `terminal_states`.

accumulator_state answers “what partial work has this handler pattern collected so far?” It tracks intermediate data for handlers using the `accumulate` pattern — which scoring dimensions have arrived, which partitions completed, what intermediate values have been computed.

`gate_state` answers “which boolean milestones are already complete?” Gates are boolean flags on the entity, set by `sets_gate` and checked by guards. They are useful for validation-style flows that track which sub-steps have finished.

These are different kinds of state that solve different problems. Blurring them together makes contracts hard to reason about.

Terminal states are absorbing. Once an entity reaches a terminal state, new events targeting it are rejected before handler execution, timers are cancelled, agent sessions are terminated, and all persisted state remains queryable. There is no mechanism to reopen a terminal entity — if the business needs to “reopen” something, the correct pattern is to create a new entity with data copied from the old one. The old entity stays terminal to preserve the audit trail.

Backward transitions to *non-terminal* states are allowed. Revision loops, reset-to-researching, and similar patterns are valid — the prohibition is specifically on leaving terminal states.

ACCUMULATION ENGINE

The accumulation engine handles the `accumulate` primitive. Think of it as “wait for enough pieces before continuing.”

When an entity first encounters a handler with `accumulate`, the platform creates an accumulator record tracking expected and received items. Each event arrival is identified by its `dedup_by` key (default: sender session ID; can be overridden to a payload field like `payload.dimension`). Duplicate arrivals with the same key are ignored — accumulation is idempotent.

After each arrival, the platform checks the completion condition:

- `all` — proceed when every expected item has been received.
- `threshold` — proceed when the received count reaches a configured number.
- `timeout` — proceed when a timer fires, regardless of how many items arrived.

When the condition is met, the handler continues to its remaining steps (`compute`, `on_complete`, etc.). If the handler declares `on_timeout`, the platform executes that branch when the timer fires before completion, with whatever partial data arrived.

AGENT SESSION MANAGEMENT

When an event arrives in an agent’s inbox, the platform creates or resumes an agent session:

1. Event arrives in inbox.
2. Prompt is loaded from `prompts/` and `{{variables}}` are substituted.
3. Tools are attached from the tool registry.
4. The agent processes the event — reasons, calls tools, emits events.

5. Each tool call is validated (schema and permissions) before execution.
6. Each emitted event is validated against its payload schema and enters the event loop.
7. Session completes when the agent signals done or hits `max_turns_per_task`.
8. Session state is persisted for audit and debugging.

The platform auto-generates emit tools from each agent's `emit_events` list. Agents do not need to list emit tools in `tools_tier2`. Universal tools (`agent_message` , `mailbox_send`) are also auto-granted.

Conversation Modes

The `conversation_mode` field controls how the platform manages session context:

- `task` — new session per event. No memory between invocations. This is the default.
- `session` — session persists across events. Conversation history accumulates.
- `session_per_entity` — one session per (agent, entity) pair. The agent retains context across multiple events for the same entity but has separate sessions for different entities. Useful for agents that build understanding over time, like a research agent assembling a brief over multiple turns.
- `stateless` — alias for `task`, normalized internally.

TIMER MODEL

Timers are durable delayed events. They are not special background magic — they are delayed event producers. When a timer fires, the resulting event goes through the normal event loop like any other event.

Timer declaration fields: `id` , `event` , `delay` (duration string or policy reference), `recurring` (boolean), `start_on` (state or event that starts the timer), `cancel_on` (state or event that cancels it).

The lifecycle: the timer starts when the entity enters the `start_on` state or the `start_on` event fires; the platform persists the timer with a `fire_at` timestamp; when `fire_at` is reached, the timer event enters the event loop; if `cancel_on` fires first, the timer is cancelled. Recurring timers restart after firing unless cancelled. Timers survive crashes because they are persisted.

EXPRESSION LANGUAGE

The platform uses CEL (Common Expression Language) for all guard checks, rule conditions, filter conditions, and `on_complete` branch conditions.

The context variables available in CEL expressions are:

- `entity` — current entity state (all fields from `entity_schema`, `state`, and `gates`)
- `payload` — current event payload fields
- `policy` — policy values from flow and root policy files
- `fan_out` — available after a `fan_out` step (e.g., `fan_out.count`)

- `accumulated` — available inside `on_complete` after accumulation

ERROR MODEL

The engine handles failures at each layer:

- **Event validation failure:** rejected, logged, not delivered, no retry.
- **Handler execution failure:** atomic boundary rolls back. Retries up to 3 with exponential backoff (1s, 2s, 4s) for transient errors. Guard failures and business logic errors are not retried. After retry exhaustion, the event goes to `dead_letters`.
- **Agent session failure:** terminated after 1 retry, then dead-lettered.
- **Timer failure:** treated as a regular event, same retry policy.
- **Chain depth overflow:** the platform tracks a `chain_depth` counter on each event. When it exceeds `50`, the would-be emitted event is intercepted and sent to `dead_letters`. The triggering handler still succeeds — chain depth overflow is an emission interception, not a handler failure.

BOOT VERIFICATION

The engine runs 19 contract verification checks at boot, after loading all contracts and before starting any nodes or agents. If any check with `error` severity fails, boot aborts. Checks with `warning` severity log a finding but allow boot to continue.

The most important checks to recognize:

- `event_chain_integrity`, `event_consumer_exists`, `event_producer_exists` — verify events have both producers and consumers.
- `payload_field_coverage`, `condition_payload_alignment` — verify handlers reference fields that actually exist.
- `state_machine_coherence` — verify state transitions make sense.
- `required_agents_match` — verify agent roles are fulfilled.
- `handler_field_compliance` — verify handlers use only valid field names and shapes.
- `single_node_per_event` — verify each event is handled by at most one system node.
- `event_cycle_detection` — verify no infinite event loops.

8. Compliance, Permissions, Tools, And Prompts

These four areas define the control layer around execution. They answer: what is an agent *allowed* to do, what tools does it have, how is its prompt constructed, and how does the platform verify all of this?

COMPLIANCE

Compliance defines both boot-time and runtime enforcement. The most important runtime checks are:

- Tool calls are validated against tool schema before execution.
- Tool calls are checked against agent permissions before execution.
- Message delivery is checked against agent messaging scope.
- State changes only follow declared `advances_to` paths.
- Guards execute before state advancement.
- Event payloads are validated before publish.
- Accumulation is idempotent.

The spec is written so both the runtime and test suite can enforce these guarantees. Compliance is not advisory — it is mechanical.

PERMISSIONS MODEL

Agents have a `permissions` list. The platform enforces permissions at tool execution time and message routing time.

The platform-defined permissions are: `agent_fire`, `agent_hire`, `agent_reconfigure`, `approve_spend`, `configure_routing`, `create_flow_instance`, `human_task_decide`, `human_task_request`, `mailbox_send`, `message_flow`, `message_peers`, `schedule`.

An agent may declare both `permissions_bundle` and explicit `permissions`. The bundle is expanded first, then explicit permissions are added (extending, not replacing).

Permission Scoping

The spec is strict about scoping. **All scoping is flow-instance-local.** No permission grants cross-flow access. Cross-flow communication happens through events only.

Messaging scope is limited to two options:

- `message_peers` — the agent can message other agents that share the same `manager_fallback` value within the same flow instance.
- `message_flow` — the agent can message any agent in the same flow instance.

There is no `message_all` and no `message_domain`. The spec explicitly rejects broader interpretations like “I can message anywhere in the product” or “I can message across flow boundaries because I know the path.”

Management scope (`agent_hire`, `agent_fire`, `agent_reconfigure`) follows the `manager_fallback` chain downward within the same flow instance. An agent can manage those below it in the chain, but cannot manage itself or its own ancestors.

Routing scope (`configure_routing`) allows self-modification and modification of agents the caller can manage.

Why `manager_fallback` Confuses People

`manager_fallback` does two things:

1. It defines escalation behavior — where dead-lettered agent sessions route to.
2. It defines the management hierarchy used by management-scope permissions.

What it does **not** do is grant general messaging access. That separation matters because otherwise hierarchy and communication scope become entangled in hard-to-audit ways. When `manager_fallback` references an agent in a parent flow, escalation produces an event through the normal event loop, not a direct message.

TOOL MODEL

The platform owns all tool schema serving. It reads tool definition YAML, generates MCP tool definitions, validates calls against schemas, and routes them to handlers.

The platform-builtin tools are:

- `create_flow_instance` — create a new dynamic flow instance from a template (**handler action only**, not directly callable by agents)
- `record_evidence` — append payload to entity accumulator (also handler action only)
- `create_entity` — create a new entity row
- `query_entities` — query entities by state, fields, or aggregation
- `agent_message` — send a message to another agent (auto-granted)
- `mailbox_send` — send an item to the human mailbox (auto-granted)

Custom tool definitions in `tools.yaml` require a `description` field and optionally include `handler_type`, `input_schema`, `output_schema`, `parameters`, and `returns`. The fields `endpoint` and `type` are explicitly not accepted.

PROMPT TEMPLATING

Agent prompts live in `prompts/{agent-id}.md`. They use `{{variable_name}}` syntax for variable substitution.

Prompt variables resolve from these sources in priority order:

1. `instance_variables` from `schema.yaml`
2. `policy` values from `policy.yaml`
3. `entity_state` fields

4. Runtime tokens (hardcoded allowlist: `current_date` , `agent_id` , `flow_instance_path`)

The substitution is intentionally simple: no logic, no conditionals, no custom mini-template engine. The platform substitutes strings from approved sources. That keeps prompt rendering easy to inspect and easy to validate.

9. Versioning

Platform and products version independently using semver.

For the platform:

- `major` — breaking changes to flow model, handler fields, boot validation, or contract formats.
- `minor` — backward-compatible new primitives, fields, or checks.
- `patch` — bug fixes and clarifications.

For products:

- `major` — breaking flow or event-schema changes.
- `minor` — additive agents, events, tools, or threshold changes.
- `patch` — prompt improvements, policy tuning, and bug fixes.

Products declare `platform_version` in `package.yaml` , and boot enforces compatibility against the running platform version.

10. Platform Tables And Runtime Data

The platform tables are the runtime's infrastructure layer. These are not contract-driven entity tables — they are platform-owned tables that exist regardless of which product contracts are loaded.

THE TABLE SET

The platform maintains these tables: `events` , `event_deliveries` , `event_receipts` , `entity_state` , `agents` , `agent_sessions` , `routing_rules` , `mailbox` , `spend_ledger` , `flow_instances` , `timers` , `dead_letters` , `schema_version` .

WHAT EACH TABLE IS FOR

Table	Purpose
events	Append-only event store
event_deliveries	Delivery tracking per subscriber
event_receipts	Per-handler completion acknowledgements and side effects
entity_state	Current entity registry and state store
agents	Runtime agent registry
agent_sessions	Session state by scope
routing_rules	Contract routes and materialized wildcard routes
mailbox	Human-in-the-loop task queue
spend_ledger	LLM usage and cost tracking
flow_instances	Static and dynamic instance registry
timers	Durable timer and scheduled task store
dead_letters	Retry exhaustion and chain depth overflow outcomes
schema_version	Applied platform schema version

The spec also defines diagnostics encoding inside existing tables, the rule that `entity_state` is the entity registry, entity metrics conventions, migration policy through `schema_version`, test bootstrap rules, and credentials policy.

For exact DDL, see platform-spec.md.

WHERE TO START

If you are onboarding and only want three tables to understand first:

- `events` — for what happened
- `entity_state` — for where things are now
- `event_receipts` — for what each handler did

Those three tables answer most runtime questions before you need the rest of the storage model.

11. Observation Model

The observation model defines what a test, audit, or flight recorder can see after handler execution.

EMITTED EVENTS

The `emitted_events` view includes events from handler `emits`, `rules`, `on_complete`, `fan_out`, and handler action side effects like `auto_emit_on_create`. It excludes agent fixture emissions, dead-lettered events, and child flow `auto_emit` events from the parent emission log.

PAYLOAD CONSTRUCTION

When `payload_transform` is absent, emitted payloads are constructed from entity fields, then overlaid with trigger payload fields, then forced platform fields. The collision priority (highest to lowest) is: platform fields > `payload_transform` > trigger payload > entity fields.

ENTITY STATE OBSERVATION

The observable entity fields after commit are: `current_state`, `gates`, `fields`, `accumulator`, `revision`.

HANDLER OUTCOME

The observable `handler_outcome` values are: `success`, `reject`, `discard`, `kill`, `escalate`, `dead_letter`, `terminal_reject`.

System node emissions appear in the triggering handler's observation chain. Agent emissions are independent events entering the loop separately.

12. Workspace Model

The workspace model defines the filesystem contract for agent sessions.

STANDARD MOUNTS

Every agent session gets exactly three standard mounts:

- `/workspace` — writable. Scoped either `per-agent` or `per-flow-instance` depending on the workspace class.
- `/data` — global, read-only. Identical across all workspace classes.
- `/opt/mas/contracts` — global, read-only. Contains the contract files.

Products cannot add new mount points. They only define workspace classes (in `policy.yaml` under `workspace_classes`) that choose the `workspace_scope` for `/workspace`. Every `workspace_class` referenced by an agent must exist in policy — missing definitions are boot errors.

This constraint is good for onboarding because every agent session has the same filesystem shape. The only variable is how widely `/workspace` is shared.

MOUNT GUARANTEES

The runtime guarantees that every agent session has all three standard mounts, that `/data` is identical across all workspace classes, that `/workspace` visibility follows the declared scope, that read-only mounts are enforced at the filesystem level, and that mount availability does not vary by `conversation_mode`, `model_tier`, or any other agent property.

DEPLOYMENT MAPPING

The same mount contract maps to local development, Docker, and production. Only the backing storage implementation changes — contracts do not. Workspace behavior is part of the platform contract, not a local-dev convenience that disappears in production.

13. Crosswalk To The Reference Spec

This guide covers every top-level section from the reference spec, reorganized into a learning order:

Spec section	Guide section
platform, vocabulary	1. Platform Identity And Vocabulary
contract_formats	3. Flow Packages And Contract Files
flow_model	4. The Flow Model
handler_specification	6. Handler Execution
engine	7. The Engine Runtime
compliance	8. Compliance
permissions_model	8. Permissions Model
tool_model	8. Tool Model
prompt_templating	8. Prompt Templating
versioning	9. Versioning
platform_tables	10. Platform Tables
observation_model	11. Observation Model
workspace_model	12. Workspace Model

14. Recommended Documentation Split

The recommended approach for this documentation:

- Keep [platform-spec.yaml](#) as the sole authority.
- Keep [platform-spec.md](#) as the exact prose rendering of that authority.
- Use this guide as the reader-first companion for humans.

That gives you three layers with clear jobs: YAML for exactness, spec markdown for exact prose reference, guide markdown for comprehension.

15. Appendix A: Runtime Story

This appendix combines the flow model, boot sequence, event loop, and handler dependency graph into one end-to-end picture.

```

contracts on disk
  └─ package.yaml, schema.yaml, nodes.yaml, events.yaml, agents.yaml, tools.yaml, policy.yaml,
prompts/
  └─ flow tree walker discovers every flow
    └─ contract merger assigns runtime paths and registries
      └─ boot validation checks pins, handlers, tools, permissions, states, prompts, and
cycles
      └─ state stores and platform tables initialize
        └─ system nodes subscribe
          └─ agents subscribe
            └─ platform ready
              └─ event arrives
                └─ payload validation
                  └─ event persisted
                    └─ subscribers resolved
                      └─ node handling
                        └─ query
                          └─ clear_gates
                            └─ guard
                              └─ accumulate
                                └─ filter
                                  └─
reduce
count
|
└─ compute
  
```



```

|                                     ↳ on_complete / rules
|
|                                     ↳ advances_to + sets_gate + data_accumulation
|
|                                     ↳ payload_transform
|
|                                     ↳ emits
|
|                                     ↳ action
|
|                                     ↳ clear
|                                     ↳ agent delivery
|                                     ↳ inbox processing, tool calls,
event emission

```

WHAT THIS RUNTIME STORY IS SAYING

The story has three distinct phases:

Phase 1: Contract loading and boot. The platform discovers flows, builds runtime addressing, resolves subscriptions, validates everything, and initializes state stores. Either everything validates or nothing starts.

Phase 2: Event routing. Every event is validated before publish, persisted before processing, serialized per entity, and delivered concurrently across entities.

Phase 3: Subscriber execution. System node handlers follow the dependency graph and commit atomically. Emitted events are delivered after commit. Agent deliveries are asynchronous.

THE MINIMAL MENTAL SHORTCUT

If someone needs the shortest faithful summary of the runtime:

- Boot builds a validated flow graph.
- Events drive all work.
- System nodes own deterministic transition logic.
- Agents own reasoning work.
- Handlers commit atomically.
- Emitted events continue the chain after commit.

If someone remembers only one sentence, it should be: “Contracts define the graph, events move work through it, and handlers commit one atomic step at a time.”

16. Appendix B: Contract Author Checklist

This checklist is derived from the boot verification list, flow coherence rules, node coherence rules, agent coherence rules, and runtime compliance expectations.

PACKAGE-LEVEL

- `package.yaml` exists.
- `schema.yaml` exists.
- The flow has either `agents.yaml` or child flows.
- Every flow declared in `package.yaml` has a matching directory under `flows/`.
- `platform_version` is compatible with the running platform version.

SCHEMA-LEVEL

- `states`, `initial_state`, and `terminal_states` form a coherent state space.
- Input pins are wired by some producer.
- Output data pins do not conflict with another flow's writes.
- `required_agents` roles are fulfilled by actual agent entries.

EVENTS

- Every emitted event has a payload schema in `events.yaml`.
- Every event in `event_handlers` appears in `subscribes_to`.
- Every event in `emit_events` has a payload schema.
- Every subscription resolves to an event some node or agent produces.
- Cross-flow event consumers reference emitter-owned schemas rather than redefining them.

NODES

- Each event is handled by at most one system node.
- `advances_to` values stay inside the flow's state space.
- Guards reference real entity or policy fields.
- `produces`, if present, matches actual emitted events. (Better to omit it and let the platform derive it.)
- Handler fields use platform-defined names and shapes only.

HANDLERS

- `on_complete` and `rules` are not used together in the same handler.
- `data_accumulation.source_event` uses the canonical name `source_event`, not `source`.

- `clear_gates` is understood as all-or-nothing entity gate reset.
- `action` is only `create_flow_instance` or `record_evidence`.
- `evidence_target` is present when `action` is `record_evidence`.
- `template`, `instance_id_from`, and `config_from` are present when `action` is `create_flow_instance`.

AGENTS

- Every tool in `tools_tier2` exists in the tool registry.
- Agent permissions cover the tools they use.
- Messaging expectations match `message_flow` and `message_peers` only.
- `manager_fallback` is treated as escalation and hierarchy metadata, not as cross-flow messaging permission.
- Every referenced `workspace_class` exists in `policy.yaml`.

PROMPTS AND POLICY

- Prompt files exist for agents that require them.
- Prompt variables resolve from allowed sources only.
- Policy keys referenced by guards or prompts exist in the merged policy tree.

RUNTIME BEHAVIOR

- Tool calls are schema-validated before execution.
- Tool calls are permission-checked before execution.
- Events are payload-validated before publish.
- Accumulation behavior is idempotent.
- State changes only occur along declared paths.
- Workspace provisioning satisfies `/workspace`, `/data`, and `/opt/mas/contracts`.

17. Appendix C: What Changes Where?

This matrix summarizes which contract file is responsible for which kind of declaration.

File	Primary responsibility	Typical contents
<code>package.yaml</code>	Flow manifest and package identity	name, version, platform_version, flows, entity_schema

File	Primary responsibility	Typical contents
<code>schema.yaml</code>	Flow contract surface	States, pins, <code>required_agents</code> , <code>initial_state</code> , <code>terminal_states</code>
<code>nodes.yaml</code>	Deterministic orchestration	<code>system_node</code> declarations, <code>subscribes_to</code> , <code>event_handlers</code> , <code>timers</code> , <code>state_schema</code> , <code>permissions</code>
<code>events.yaml</code>	Event payload schemas	Event names and payload field types
<code>agents.yaml</code>	Agent registry	Subscriptions, <code>emit_events</code> , <code>tools</code> , <code>permissions</code> , model and conversation settings
<code>tools.yaml</code>	Tool schema definitions	Custom tool descriptions, input and output schemas, handler type
<code>policy.yaml</code>	Flow-scoped configuration	Policy keys, bundles, <code>workspace_classes</code>
<code>prompts/</code>	Agent prompt bodies	<code>prompts/{agent-id}.md</code> files with <code>{{variable}}</code> placeholders

PRACTICAL AUTHORIZING ORDER

When authoring a new flow, the least confusing sequence is:

1. Define the flow lifecycle in `schema.yaml` — states, pins, required roles.
2. Define event payload shapes in `events.yaml`.
3. Define deterministic transitions in `nodes.yaml`.
4. Define required and concrete agents in `schema.yaml` and `agents.yaml`.
5. Define tool schemas in `tools.yaml`.
6. Define thresholds, bundles, and workspace classes in `policy.yaml`.
7. Write prompts in `prompts/`.

THE COMMON MISPLACEMENTS

The spec repeatedly draws these boundaries:

- Event routing is not declared in `events.yaml` — it is derived from subscriptions.
- Tool endpoint configuration is not part of `tools.yaml`.
- Cross-flow messaging is not granted by permissions.
- `manager_fallback` is not a direct messaging grant.
- `create_flow_instance` is not an agent-callable tool — it is a handler action.

This appendix is worth revisiting during onboarding because most contract mistakes are not syntax errors. They are ownership mistakes: putting a concept in the wrong file or assuming one field does more than the spec says it does.

18. Appendix D: State And Storage Cheat Sheet

This appendix groups the runtime state and the platform tables by the questions operators and developers usually ask.

“WHAT STATE DOES AN ENTITY HAVE?”

Three persisted state kinds per entity instance:

- `entity_state` — lifecycle position in `current_state`, gate flags in `gates`, contract-visible fields in `fields`.
- `accumulator_state` — intermediate data for handlers using `accumulate`, stored in per-node state tables.
- `gate_state` — boolean milestone flags, stored in `entity_state.gates`.

“WHERE DO I LOOK FOR EVENT HISTORY?”

- `events` — raw event history (append-only)
- `event_deliveries` — per-subscriber delivery status
- `event_receipts` — per-handler outcomes and side effects
- `dead_letters` — retry-exhausted and chain-depth-overflow failures

“WHERE DO I LOOK FOR FLOW ROUTING?”

`routing_rules` contains both contract-defined routing patterns (inserted at boot) and materialized concrete routes for dynamic instances (inserted by `create_flow_instance`). Instance termination inactivates materialized rows.

“WHERE DO I LOOK FOR SESSIONS AND AGENTS?”

- `agents` — current runtime agent registry
- `agent_sessions` — session state, keyed by `(agent_id, scope_key)` with entity, flow, or global scope

"WHERE DO I LOOK FOR HUMAN TASKS?"

`mailbox` stores `human_task`, `review_request`, `approval`, `alert`, and `operational_decision` items.

"WHERE DO I LOOK FOR SPEND?"

`spend_ledger` stores one row per agent invocation, tracking `input_tokens`, `output_tokens`, `cost_usd`, `model`, and `invocation_type`.

"WHERE DO I LOOK FOR DYNAMIC FLOW INSTANCES?"

`flow_instances` tracks `instance_id`, `flow_template`, `mode`, `parent_instance`, `config`, and `status`.

"WHERE DO I LOOK FOR TIMERS?"

`timers`.